

Lab Notebook #9:

XAI Analysis using SHAP and LIME

XAI

Explainable AI (XAI) has become an essential area in artificial intelligence, especially given increasing calls for making machine learning models more understandable and transparent.

Why Explainability in AI?

AI models, especially complex ones like deep neural networks, often operate as “black boxes,” meaning their internal workings are hard to understand, even for experts. While they might make accurate predictions or decisions, it’s often unclear why a particular decision was made. This lack of transparency can raise issues related to trust, accountability, and fairness:

- **Trust:** Users are more likely to trust AI systems if they can understand their reasoning.
- **Accountability:** Organizations need to explain AI decisions to ensure they align with ethical and legal standards, especially in critical areas like healthcare and criminal justice.
- **Fairness and Bias:** Understanding an AI’s decision-making process can reveal and help mitigate biases.

XAI techniques are typically divided into two types: global explanations and local explanations.

1. Local explanations focus on understanding specific predictions. They are useful for explaining why a model made a particular decision, for instance, why a certain loan application was rejected.
 2. Global explanations aim to explain the entire model’s behavior. They’re good for understanding how the model works in general because they offer insights into which features (or input variables) are most important across all predictions.
- **SHAP** is based on cooperative game theory and assigns each feature an importance value for a given prediction. It’s particularly useful for interpreting complex models because it calculates the so-called Shapley value, which measures a feature’s contribution by considering all possible feature combinations. This makes SHAP globally consistent and theoretically grounded.
 - **LIME** operates by creating a local surrogate model around each prediction, perturbing input features slightly to assess their influence on the output. It then builds a simpler, interpretable model (like linear regression) on these perturbed points to approximate the complex model’s behavior around that specific instance. LIME is model-agnostic and provides highly interpretable explanations but is less theoretically grounded than SHAP.

XAI in Python

In this lab, we'll use SHAP (SHapley Additive exPlanations) and LIME (Local Interpretable Model-agnostic Explanations) to understand how the model we built last week helps predict 30-day readmissions for diabetes patients.

Install packages

```
!pip install shap lime scikit-learn tensorflow pandas matplotlib seaborn
```

```
Requirement already satisfied: cloudpickle in /usr/local/lib/python3.12/dist-packages (from shap) (3.1.1)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.12/dist-packages (from shap) (4.15.0)
Requirement already satisfied: scikit-image>=0.12 in /usr/local/lib/python3.12/dist-packages (from lime) (0.25.2)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.4.0)
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (25.9.23)
Requirement already satisfied: gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.6.0)
Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.2.0)
```

```

Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.60.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.4.9)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.2.5)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from astunparse==1.6.0->tensorflow)
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (13.9.4)
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (0.1.0)
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (0.17.0)
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /usr/local/lib/python3.12/dist-packages (from numba>=0.54->shap) (
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->t
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorf
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorf
Requirement already satisfied: networkx>=3.0 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (3.5)
Requirement already satisfied: imageio!=2.35.0,>=2.33 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime)
Requirement already satisfied: tifffile>=2022.8.12 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (
Requirement already satisfied: lazy-loader>=0.4 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (0.4
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0->tensorflow)
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from tensorboar
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0->tensorflow)
Requirement already satisfied: MarkupSafe>=2.1.1 in /usr/local/lib/python3.12/dist-packages (from werkzeug>=1.0.1->tensorboard-
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras>=3.5.0->tensc
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras>=3.5.0->ter
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich->keras>=
Building wheels for collected packages: lime
  Building wheel for lime (setup.py) ... done
  Created wheel for lime: filename=lime-0.2.0.1-py3-none-any.whl size=283834 sha256=a7c8d032210bf924e7c0779f55ad41919cd51bc3fe
  Stored in directory: /root/.cache/pip/wheels/e7/5d/0e/4b4fff9a7468fed5633211fb3b76d1db43fe806a17fb7486a
Successfully built lime
- . . . . .

```

✦ Import and set up packages

```

!pip install lime
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import shap
import lime
import lime.lime_tabular
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, roc_auc_score
from sklearn.pipeline import Pipeline
from sklearn.utils import Bunch
from sklearn.metrics import (
    balanced_accuracy_score,
    roc_auc_score,
    accuracy_score,
    recall_score,
    confusion_matrix,
    roc_auc_score,
    roc_curve,
    RocCurveDisplay)

# Set display options
pd.set_option("display.float_format", "{:.2f}".format)
sns.set()

```

```

Requirement already satisfied: lime in /usr/local/lib/python3.12/dist-packages (0.2.0.1)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from lime) (3.10.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from lime) (2.0.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from lime) (1.16.2)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from lime) (4.67.1)
Requirement already satisfied: scikit-learn>=0.18 in /usr/local/lib/python3.12/dist-packages (from lime) (1.6.1)
Requirement already satisfied: scikit-image>=0.12 in /usr/local/lib/python3.12/dist-packages (from lime) (0.25.2)
Requirement already satisfied: networkx>=3.0 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (3.5)
Requirement already satisfied: pillow>=10.1 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (11.3.0)
Requirement already satisfied: imageio!=2.35.0,>=2.33 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime)
Requirement already satisfied: tifffile>=2022.8.12 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (2
Requirement already satisfied: packaging>=21 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (25.0)
Requirement already satisfied: lazy-loader>=0.4 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (0.4)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.18->lime) (1.5.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.18->lime) (
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (4.60.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (1.4.9)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (3.2.5)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (2.9.0.po

```

```
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib->lime)
```

Dataset and task

We'll be working with the same clinical dataset that tracks hospital readmissions for diabetes patients over a ten-year period (1998-2008) from last time.

As a reminder, this dataset includes information on patient demographics, diagnoses, diabetes medications, number of prior hospital visits, payer details, and whether a patient was readmitted within 30 days after release.

Just like last time, our goal is to build a classification model that predicts whether patients should be referred to their primary care physicians for a high-risk care management program. A positive prediction would mean a recommendation for enrollment in this program.

Load the dataset

Let's load the dataset and review it like we did last time.

```
df = pd.read_csv("https://respai.s3.us-east-2.amazonaws.com/fairlearn_data_preprocessed.csv")
```

This dataset has been used by [Strack et al. \(2014\)](#) to model the relationship between patient readmission and HbA1c measurement during admission, based on primary medical diagnosis.

The dataset is publicly available through the UCI Machine Learning Repository.

df

	race	gender	age	discharge_disposition_id	admission_source_id	time_in_hospital	medical_specialty	num_
0	Caucasian	Female	30 years or younger	Other	Referral	1	Other	
1	Caucasian	Female	30 years or younger	Discharged to Home	Emergency	3	Missing	
2	AfricanAmerican	Female	30 years or younger	Discharged to Home	Emergency	2	Missing	
3	Caucasian	Male	30-60 years	Discharged to Home	Emergency	2	Missing	
4	Caucasian	Male	30-60 years	Discharged to Home	Emergency	1	Missing	
...	...	...	...	...	...	...	...	...
101761	AfricanAmerican	Male	Over 60 years	Other	Emergency	3	Missing	
101762	AfricanAmerican	Female	Over 60 years	Other	Other	5	Missing	
101763	Caucasian	Male	Over 60 years	Discharged to Home	Emergency	1	Missing	
101764	Caucasian	Female	Over 60 years	Other	Emergency	10	Other	
101765	Caucasian	Male	Over 60 years	Discharged to Home	Emergency	6	Missing	

101766 rows × 25 columns

```
categorical_values = {}
for col in df:
    if col not in {'time_in_hospital', 'num_lab_procedures',
                  'num_procedures', 'num_medications', 'number_diagnoses'}:
        categorical_values[col] = pd.Series(df[col].value_counts().index.values)
categorical_values_df = pd.DataFrame(categorical_values).fillna('')
categorical_values_df.T
```

	0	1	2	3	4	5
race	Caucasian	AfricanAmerican	Unknown	Hispanic	Other	Asian
gender	Female	Male	Unknown/Invalid			
age	Over 60 years	30-60 years	30 years or younger			
discharge_disposition_id	Discharged to Home	Other				
admission_source_id	Emergency	Referral	Other			
medical_specialty	Missing	Other	InternalMedicine	Emergency/Trauma	Family/GeneralPractice	Cardiology
primary_diagnosis	Other	Respiratory Issues	Diabetes	Genitourinary Issues	Musculoskeletal Issues	
max_glu_serum	Norm	>200	>300			
A1Cresult	>8	Norm	>7			
insulin	No	Steady	Down	Up		
change	No	Ch				
diabetesMed	Yes	No				
medicare	False	True				
medicaid	False	True				
had_emergency	False	True				
had_inpatient_days	False	True				
had_outpatient_days	False	True				
readmitted	NO	>30	<30			

Save a list of all categorical features:

```
categorical_features = [
    "race",
    "gender",
    "age",
    "discharge_disposition_id",
    "admission_source_id",
    "medical_specialty",
    "primary_diagnosis",
    "max_glu_serum",
    "A1Cresult",
    "insulin",
    "change",
    "diabetesMed",
    "readmitted"
]
```

```
for col_name in categorical_features:
    df[col_name] = df[col_name].astype("category")
```

▼ Data Preprocessing

Normalized sensitive attribute frequencies from last time:

```
df["race"].value_counts(normalize=True)
```

proportion	
race	
Caucasian	0.75
AfricanAmerican	0.19
Unknown	0.02
Hispanic	0.02
Other	0.01
Asian	0.01

dtype: float64

In this dataset, patients are predominantly *Caucasian* (75%). The next largest racial group is *AfricanAmerican*, making up 19% of the patients. The remaining race categories (including *Unknown*) compose only 6% of the data.

```
# retain the original race as race_all, and merge Asian+Hispanic+Other
df["race_all"] = df["race"]
df["race"] = df["race"].replace({"Asian": "Other", "Hispanic": "Other", "Unknown": "Other"})

# drop gender group Unknown/Invalid
df = df.query("gender != 'Unknown/Invalid'")
```

```
/tmp/ipython-input-832111096.py:3: FutureWarning: The behavior of Series.replace (and DataFrame.replace) with CategoricalDtype is deprecated, in favor of using .take with masked values in a Series.
df["race"] = df["race"].replace({"Asian": "Other", "Hispanic": "Other", "Unknown": "Other"})
```

✓ Training the initial models

✓ Prepare training and test datasets

As we mentioned in the task definition, our target variable is **readmission within 30 days**, and our sensitive feature is **race**.

```
target_variable = "readmit_30_days"
demographic = ["race", "gender"]
sensitive = ["race"]
```

```
Y = df.loc[:, target_variable]
```

YOUR TASK: Print the normalized frequencies of the target variable

```
#Your code:
df["readmit_30_days"].value_counts(normalize=True)
```

proportion	
readmit_30_days	
0	0.89
1	0.11

dtype: float64

We next drop the features we won't use in the model and expand the categorical features into 0/1 indicators ("dummies").

```
X = pd.get_dummies(df.drop(columns=[
    "race",
    "race_all",
    "discharge_disposition_id",
    "readmitted",
    "readmit_binary",
    "readmit_30_days"
]))
```

```
X.head() # sanity check
```

	time_in_hospital	num_lab_procedures	num_procedures	num_medications	number_diagnoses	medicare	medicaid	had_emergency	ha
0	1	41	0	1	1	False	False	False	ha
1	3	59	0	18	9	False	False	False	ha
2	2	11	5	13	6	False	False	False	ha
3	2	44	1	16	7	False	False	False	ha
4	1	51	0	8	5	False	False	False	ha

5 rows × 44 columns

```
random_seed = 123
np.random.seed(random_seed)
```

```
X_train, X_test, Y_train, Y_test, df_train, df_test = train_test_split(
    X,
    Y,
    df,
    test_size=0.30,
    stratify=Y,
    random_state=random_seed
)
```

len(X_train)

71234

Our performance metric is **balanced accuracy** (sensitivity + specificity/2), so for the purposes of training (but not evaluation!) we will resample the data set, so that it has the same number of positive and negative examples.

- Sensitivity/ Recall for the positive class = True Positives (TP)/ True Positives (TP)+False Negatives (FN)
- Specificity/ Recall for the negative class = True Negatives (TN)/ True Negatives (TN)+False Positives (FP)

Because we are downsampling the number of negative examples, we create a training dataset with a significantly lower number of data points. For more complex machine learning models, this lower number of training data points may affect the model's accuracy.

```
def resample_dataset(X_train, Y_train):

    negative_ids = Y_train[Y_train == 0].index
    positive_ids = Y_train[Y_train == 1].index
    #balancing the train set by picking random samples
    balanced_ids = positive_ids.union(np.random.choice(a=negative_ids, size=len(positive_ids)))

    X_train = X_train.loc[balanced_ids, :]
    Y_train = Y_train.loc[balanced_ids]
    return X_train, Y_train
```

X_train_bal, Y_train_bal = resample_dataset(X_train, Y_train)

len(X_train_bal)

15900

✓ Save descriptive statistics of training and test data

We next evaluate the descriptive statistics of the training dataset. These can be provided as part of *model cards* to document our training.

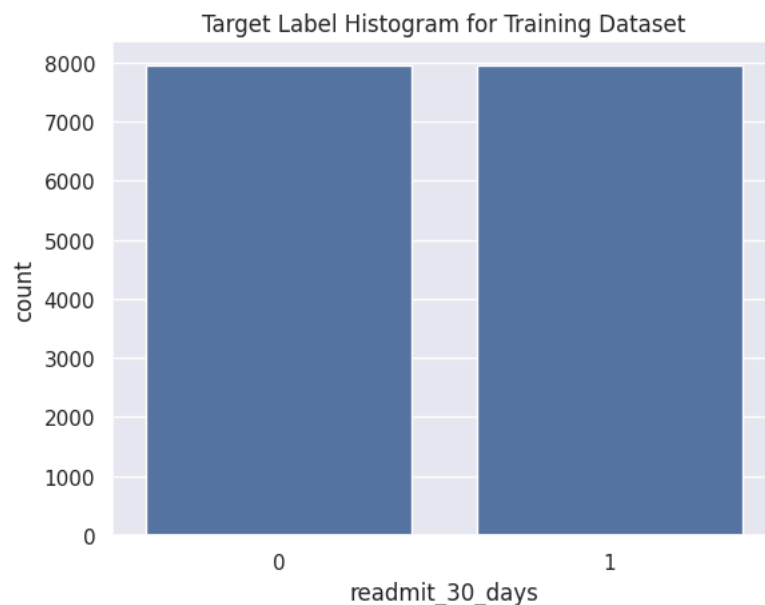
```
sns.countplot(x=Y_test)
plt.title("Target Label Histogram for Test Dataset")
```

```
Text(0.5, 1.0, 'Target Label Histogram for Test Dataset')
```



```
sns.countplot(x=Y_train_bal)  
plt.title("Target Label Histogram for Training Dataset")
```

```
Text(0.5, 1.0, 'Target Label Histogram for Training Dataset')
```

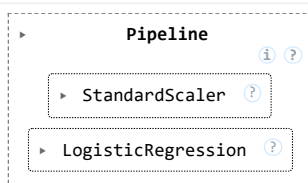


✓ Train the logistic regression model

We train a logistic regression model and save its predictions on test data for analysis.

```
pipeline = Pipeline(steps=[  
    ("preprocessing", StandardScaler()),  
    ("logistic_regression", LogisticRegression(max_iter=1000))  
)
```

```
pipeline.fit(X_train_bal, Y_train_bal)
```



```
Y_pred_proba = pipeline.predict_proba(X_test)[: ,1]
Y_pred = pipeline.predict(X_test)
```

Y_pred

array([0, 0, 0, ..., 0, 0, 0])

Y_test

readmit_30_days	
4762	0
19503	0
39297	0
68521	0
97908	0
...	...
94066	0
14567	0
46504	0
46999	0
36144	0

30529 rows × 1 columns

dtype: int64

Check model performance on test data.

```
# Show balanced accuracy rate
balanced_accuracy_score(Y_test, Y_pred)
```

np.float64(0.5920199484757553)

Y_test[:10]

readmit_30_days	
4762	0
19503	0
39297	0
68521	0
97908	0
27148	0
60207	0
39449	0
47851	0
833	0

dtype: int64

Y_pred[:10]

array([0, 0, 0, 0, 0, 0, 0, 1, 0, 0])

X_test

	time_in_hospital	num_lab_procedures	num_procedures	num_medications	number_diagnoses	medicare	medicaid	had_emergency
4762	3	23	2	10	9	False	False	False
19503	4	59	1	18	8	False	False	False
39297	4	40	1	25	5	True	False	False
68521	5	63	2	21	7	False	False	False
97908	3	31	0	13	7	False	False	True
...	...	...	...	...	...	...	...	...
94066	8	47	0	27	9	True	False	False
14567	4	79	0	18	8	False	False	False
46504	4	32	5	20	8	False	False	False
46999	3	11	0	8	9	True	False	False
36144	2	22	0	6	6	False	False	False

30529 rows × 44 columns

```
X_train_bal = X_train_bal.replace({True: 1, False: 0})
X_train_bal = X_train_bal.astype(int)
```

```
X_test = X_test.replace({True: 1, False: 0})
X_test = X_test.astype(int)
```

```
/tmp/ipython-input-1896363500.py:1: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version.
  X_train_bal = X_train_bal.replace({True: 1, False: 0})
/tmp/ipython-input-1896363500.py:4: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version.
  X_test = X_test.replace({True: 1, False: 0})
```

X_test

	time_in_hospital	num_lab_procedures	num_procedures	num_medications	number_diagnoses	medicare	medicaid	had_emergency
4762	3	23	2	10	9	0	0	0
19503	4	59	1	18	8	0	0	0
39297	4	40	1	25	5	1	0	0
68521	5	63	2	21	7	0	0	0
97908	3	31	0	13	7	0	0	1
...	...	...	...	...	...	...	...	...
94066	8	47	0	27	9	1	0	0
14567	4	79	0	18	8	0	0	0
46504	4	32	5	20	8	0	0	0
46999	3	11	0	8	9	1	0	0
36144	2	22	0	6	6	0	0	0

30529 rows × 44 columns

✓ Applying LIME and SHAP for Explainability on the Logistic Regression Model

Next, let's take a look at how we can apply LIME to Explain the same model.

LIME explains individual predictions by creating local surrogate models which makes it effective for interpreting single-instance decisions.

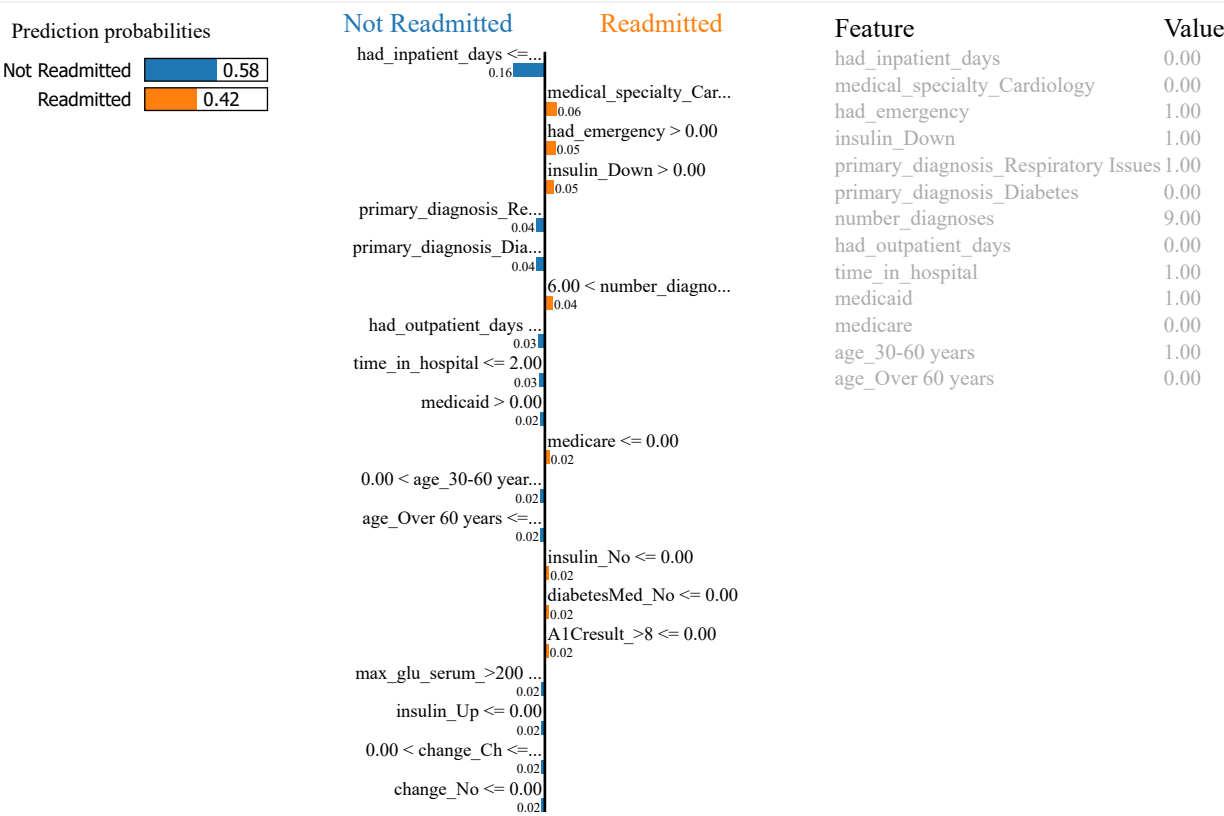
```
from lime.lime_tabular import LimeTabularExplainer

# Initialize LIME explainer with training data
explainer_lime_lr = LimeTabularExplainer(X_train_bal.values,
                                         feature_names=X_train_bal.columns,
                                         class_names=["Not Readmitted", "Readmitted"],
                                         discretize_continuous=True)

# Generate LIME explanation for a single instance (e.g., first row in X_test)
#Checking for Patient no 7:
lime_exp_lr = explainer_lime_lr.explain_instance(X_test.iloc[6].values, pipeline.predict_proba, num_features=20)
```

/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but warnings.warn(

```
# Display LIME explanation
lime_exp_lr.show_in_notebook()
```



```
X_test.iloc[6]
```

	60207
time_in_hospital	1
num_lab_procedures	50
num_procedures	0
num_medications	13
number_diagnoses	9
medicare	0
medicaid	1
had_emergency	1
had_inpatient_days	0
had_outpatient_days	0
gender_Female	1
gender_Male	0
gender_Unknown/Invalid	0
age_30 years or younger	0
age_30-60 years	1
age_Over 60 years	0
admission_source_id_Emergency	1
admission_source_id_Other	0
admission_source_id_Referral	0
medical_specialty_Cardiology	0
medical_specialty_Emergency/Trauma	0
medical_specialty_Family/GeneralPractice	0
medical_specialty_InternalMedicine	1
medical_specialty_Missing	0
medical_specialty_Other	0
primary_diagnosis_Diabetes	0
primary_diagnosis_Genitourinary Issues	0
primary_diagnosis_Musculoskeletal Issues	0
primary_diagnosis_Other	0
primary_diagnosis_Respiratory Issues	1
max_glu_serum_>200	0
max_glu_serum_>300	0
max_glu_serum_Norm	0
A1Cresult_>7	0
A1Cresult_>8	0
A1Cresult_Norm	0
insulin_Down	1
insulin_No	0
insulin_Steady	0
insulin_Up	0
change_Ch	1
change_No	0
diabetesMed_No	0
diabetesMed_Yes	1

dtype: int64

YOUR TASK: Plot the same analysis for the 105th observation in the test set and comment briefly on what you see.

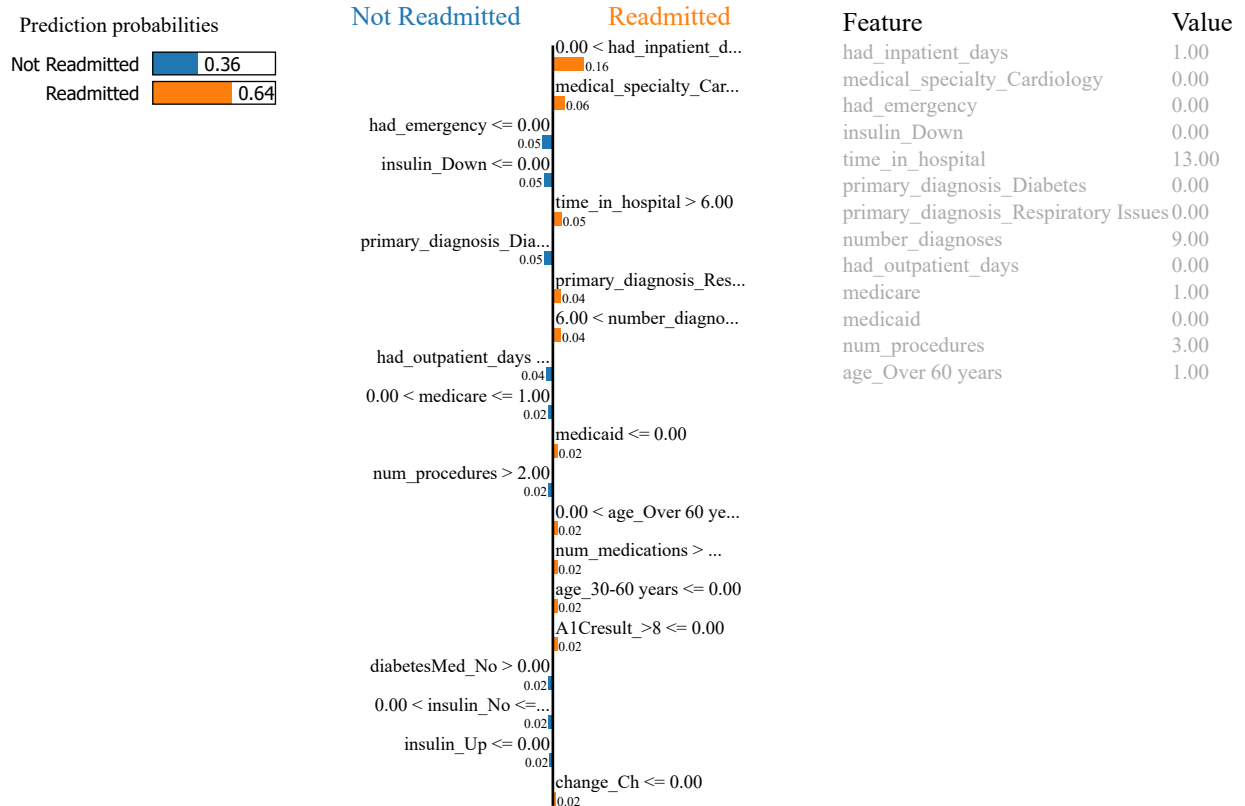
```
#Enter your code here:
```

```
lime_exp_lr = explainer_lime_lr.explain_instance(X_test.iloc[104].values, pipeline.predict_proba, num_features=20)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but warnings.warn(
```

```
# Display LIME explanation
```

```
lime_exp_lr.show_in_notebook()
```



Needs hospital care as readmitted shows a probability of 64%

What's the LIME process workflow?

- LIME is set up with the training data to learn about feature distributions and interactions.
- The `.explain_instance` method perturbs the values of the selected instance (e.g., `X_test.iloc[0]`) to see how small changes impact predictions, which allows it to approximate the model's local behavior.
- LIME returns an interpretable view of how each feature influenced the prediction, visualized directly within the notebook.

Let's now take a look at SHAP.

SHAP uses a method called Explainer for model-agnostic explainability, which is compatible with any model, including logistic regression.

```
import shap

# Extract the logistic regression model from the pipeline
logistic_model = pipeline.named_steps["logistic_regression"]

# Initialize SHAP LinearExplainer for logistic regression
explainer_shap_lr = shap.LinearExplainer(logistic_model, X_train_bal)

# Calculate SHAP values for the test data
shap_values_lr = explainer_shap_lr.shap_values(X_test)

shap_values_lr
```

```
array([[ -0.14571691,  0.54370401, -0.03002058, ..., -0.01773029,
         0.00862965,  0.00862965],
       [ -0.0557682 , -0.40877017,  0.01412733, ...,  0.0145066 ,
        -0.02889058, -0.02889058],
       [ -0.0557682 ,  0.09392454,  0.01412733, ..., -0.01773029,
         0.00862965,  0.00862965],
       ...,
       [ -0.0557682 ,  0.30558547, -0.16246433, ...,  0.0145066 ,
        -0.02889058, -0.02889058],
       [ -0.14571691,  0.8611954 ,  0.05827525, ...,  0.0145066 ,
```

```
0.00862965, 0.00862965],
[-0.23566562, 0.57016163, 0.05827525, ..., 0.0145066 ,
-0.02889058, -0.02889058]])
```

Each patient is associated with a list of values, one value for each variable in the dataset. Positive sign means the patient needs readmission and negative means they do not medical care. The number represents the magnitude. The entire output is a dataframe.

```
len(shap_values_lr)
```

```
30529
```

```
len(Y_test)
```

```
30529
```

```
shap_values_lr[[0]]
```

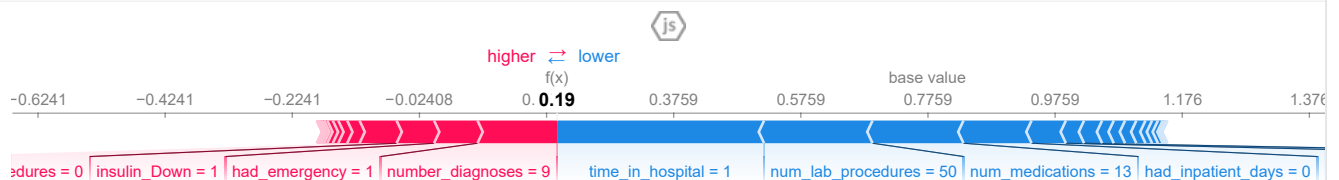
```
array([[ -1.45716909e-01,  5.43704010e-01, -3.00205819e-02,
        -2.78009323e-01,  1.25231556e-01,  1.40821009e-02,
         1.85363242e-04, -1.11291672e-02, -1.07115989e-01,
        -1.03444912e-02, -1.56797809e-03, -1.56797809e-03,
         0.00000000e+00,  1.18159027e-04, -2.85946011e-02,
        -2.83914856e-02, -4.65359717e-03,  7.27551607e-04,
         1.12914721e-03,  5.41323694e-04,  7.93951227e-05,
         3.80700726e-04, -2.32912171e-04, -1.44090728e-02,
        -2.11347212e-04, -7.45005085e-03,  4.79754999e-04,
        -1.23538014e-04,  5.40864173e-03,  5.99967715e-03,
         0.00000000e+00,  0.00000000e+00, -6.15859608e-05,
         4.39338716e-04,  3.88144239e-03,  2.75393464e-05,
        -7.28515110e-03,  1.93791329e-02, -1.11662111e-02,
        -3.59261244e-03, -1.77302870e-02, -1.77302870e-02,
         8.62965273e-03,  8.62965273e-03]])
```

Visualize SHAP values for a specific instance and as a summary plot

```
Y_pred[:10]
```

```
array([0, 0, 0, 0, 0, 0, 0, 1, 0, 0])
```

```
shap.initjs()
shap.force_plot(explainer_shap_lr.expected_value, shap_values_lr[6], X_test.iloc[6])
```

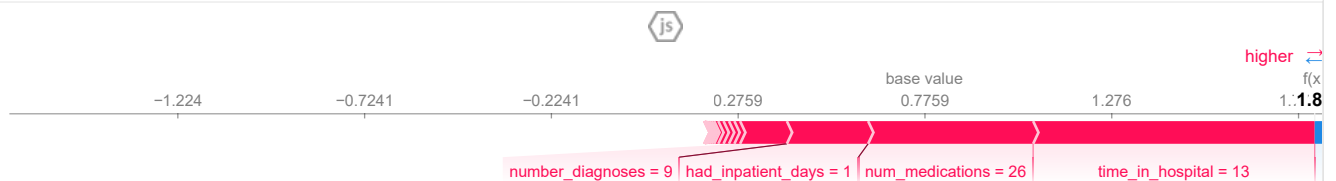


In SHAP, the base value reflects the average model prediction when no information about any particular instance is available. This value is often close to the average probability of the positive class across all instances in the dataset. For example, if the dataset has a 30% overall probability of patients being readmitted, the base value would be around that percentage.

Since we're using a logistic regression model, the SHAP base value is actually in log-odds.

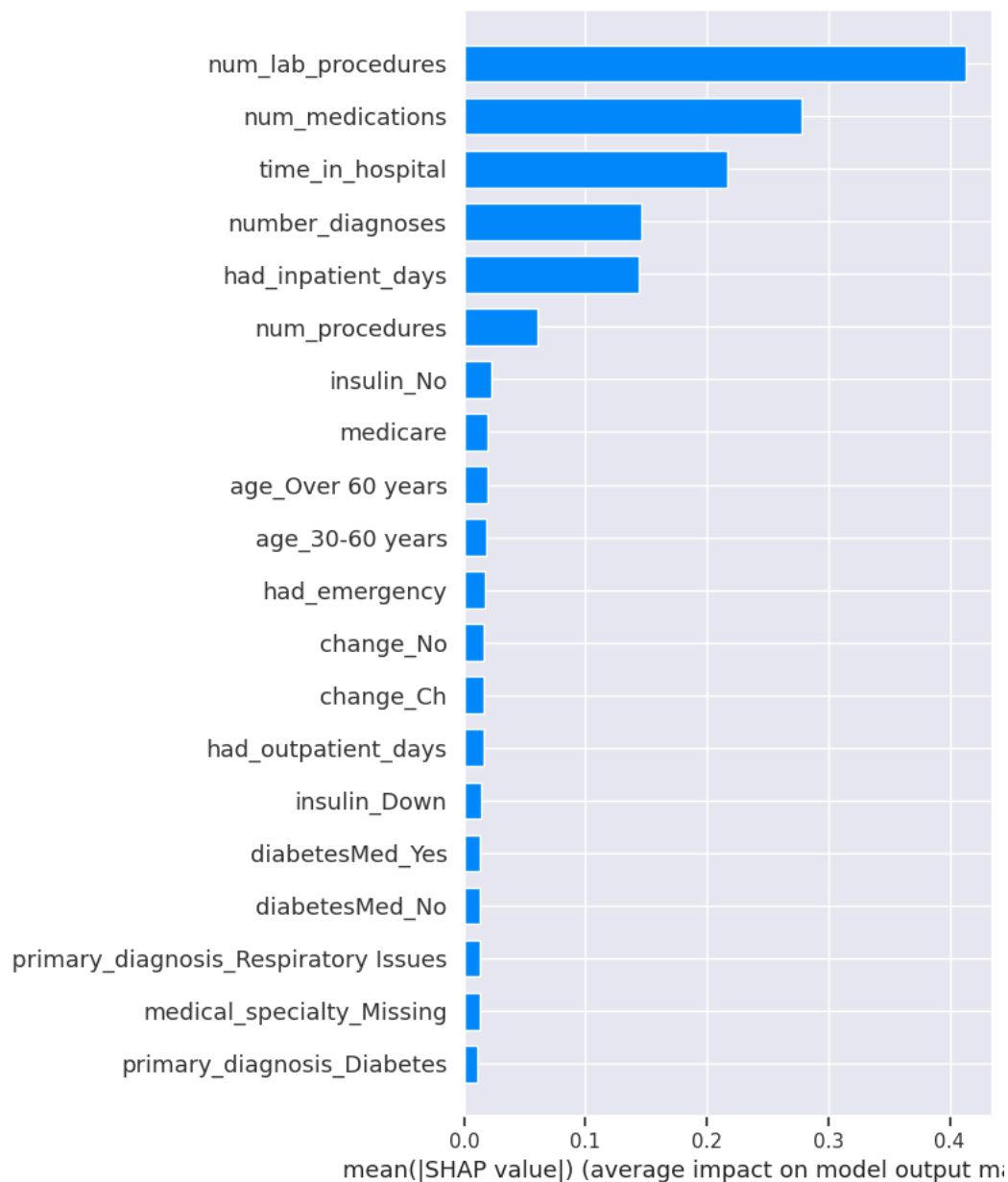
YOUR TASK: Create a SHAP force plot for the 105th observation in the test set.

```
#Enter your code here:
shap.initjs()
shap.force_plot(explainer_shap_lr.expected_value, shap_values_lr[104], X_test.iloc[104])
```



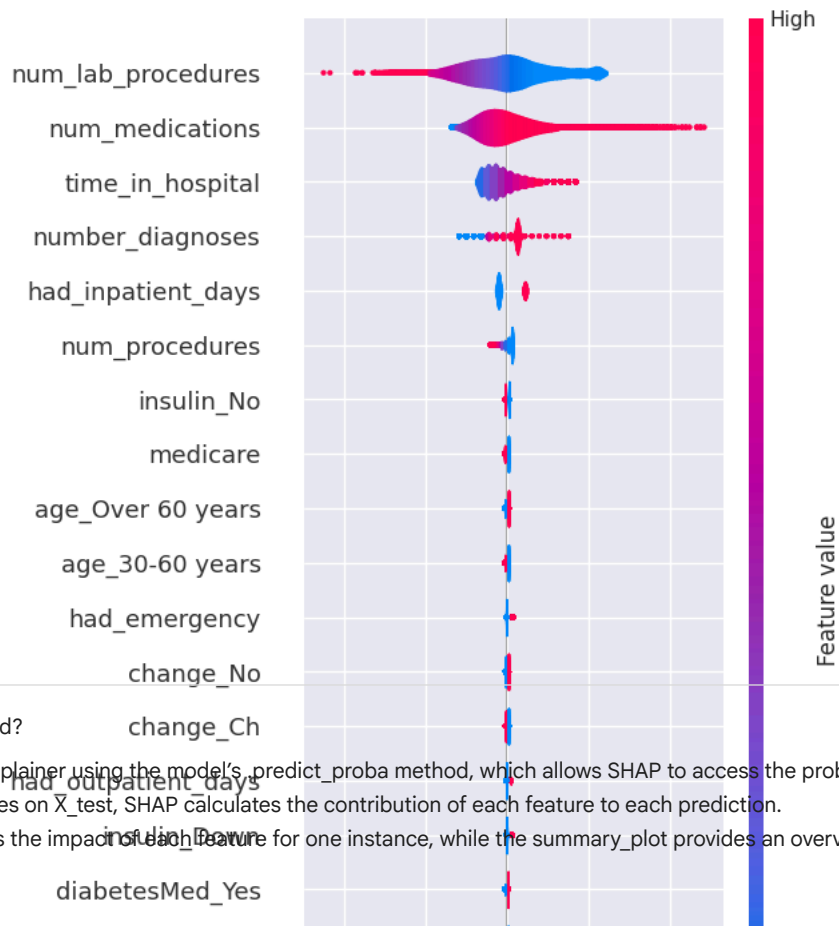
```
# SHAP Summary Plot for overall impact
shap.summary_plot(shap_values_lr, X_test, plot_type="bar")
```

```
/tmp/ipython-input-3498952951.py:2: FutureWarning: The NumPy global RNG was seeded by calling `np.random.seed`. In a future ver
shap.summary_plot(shap_values_lr, X_test, plot_type="bar")
```



```
shap.summary_plot(shap_values_lr, X_test, plot_type="violin")
```

```
/tmp/ipython-input-3220868778.py:1: FutureWarning: The NumPy global RNG was seeded by calling `np.random.seed`. In a future ver
shap.summary_plot(shap_values_lr, X_test, plot_type="violin")
```



What are the steps involved?

- We create a SHAP explainer using the model's `predict_proba` method, which allows SHAP to access the probability predictions.
- By calling `.shap_values` on `X_test`, SHAP calculates the contribution of each feature to each prediction.
- The `force_plot` shows the impact of each feature for one instance, while the `summary_plot` provides an overview across all instances in `X_test`.